

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.5.25

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1030 求先序排列	普及	二叉树重建、递归
P1305 新二叉树	普及	二叉树构建、前序遍历
P1229 遍历问题	普及/提高-	二叉树遍历性质、组合计数
P1271 选举学生会	普及	计数排序、桶排序

（一）求先序排列

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1030>

知识点：二叉树重建、递归

题目简述

给出一棵二叉树的中序遍历和后序遍历，要求输出它的先序遍历。

题目保证结点值互不相同，且结点总数不超过 8。

1.00s, 128MB。

题目分析

1. 读题

已知中序和后序，求先序，是经典的二叉树重建问题。题目并不一定要求显式把整棵树结构建出来，只要能按照正确顺序输出先序遍历即可。

2. 性质观察

后序遍历的最后一个结点一定是当前子树的根。找到根之后，就可以在中序遍历中把整棵树分成左子树和右子树两部分，再递归处理。

而先序遍历的顺序是：

1. 根；
2. 左子树；
3. 右子树。

3. 程序设计思路

递归处理每一段中序和后序区间：

1. 当前后序区间最后一个字符就是根；
2. 在中序区间中找到根的位置；
3. 由此得到左子树大小和右子树大小；
4. 先输出根；
5. 再递归处理左子树；
6. 最后递归处理右子树。

当前代码并没有显式建出树结构，而是直接在字符串区间上递归：每次找到根在中序中的位置后，立刻输出根字符，再递归处理左右两段子串。

4. 数据结构与算法选择

本题使用递归最自然。当前实现直接传递中序串、后序串的首地址和长度，不需要额外开结点数组，代码会更精炼。

5. 时空复杂度分析

- 时间复杂度：朴素做法为 $O(n^2)$ ，因为每次都可能在线性时间内查找根位置。
- 空间复杂度： $O(n)$ ，主要来自递归栈深度。

由于 $n \leq 8$ ，即使使用最朴素的递归实现也完全足够。

代码实现

洛谷 / 评测记录 / 评测详情

R281462738 记录详情

编程语言

C O2

代码长度

459B

用时

16ms

内存

812.00KB

测试点信息

源代码

测试点信息

#1

AC
3ms/788.00KB

#2

AC
3ms/788.00KB

#3

AC
3ms/788.00KB

#4

AC
4ms/812.00KB

#5

AC
3ms/812.00KB

jsj0708Baijiahui

所属题目

P1030 [NOIP 2001 普及组] 求先...

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:47:36

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int getPreOrder(char* in, char* post, int len)
{
    if (len <= 0)
        return 0;
    char root = post[len - 1];
    int pos = 0;
    while (in[pos] != root)
    {
        pos++;
    }
    printf("%c", root);
    getPreOrder(in, post, pos);
    getPreOrder(in + pos + 1, post + pos, len - pos - 1);
}
int main()
{
    char in[10], post[10];
    scanf("%s%s", in, post);
    int n = strlen(in);
    getPreOrder(in, post, n);
    printf("\n");
    return 0;
}

```

(二) 新二叉树

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1305>

知识点：二叉树构建、前序遍历

题目简述

输入一棵二叉树的信息。每行给出一个结点以及它的左右儿子，空结点用 * 表示，并保证输入的第一行结点就是根节点。

要求输出整棵树的前序遍历结果。

对于全部数据， $1 \leq n \leq 26$ 。

1.00s, 128MB。

题目分析

1. 读题

题目已经直接给出了每个结点的左右孩子，因此不需要从遍历序列反推结构，只要把这棵树按输入信息存下来，再做一次前序遍历即可。

2. 性质观察

前序遍历顺序固定为：

1. 访问根结点；
2. 遍历左子树；
3. 遍历右子树。

由于结点编号使用字母，而且结点总数很小，可以直接用数组或映射保存每个字母结点的左右孩子。

3. 程序设计思路

1. 读入 n 行结点信息；
2. 记录每个结点的左儿子和右儿子；
3. 输入第一行给出的结点就是根；
4. 从根结点开始递归进行前序遍历；
5. 遇到 * 表示空结点，直接返回。

当前代码中把每个字母结点映射成 $0..25$ 的数组下标，使用 `tree[27]` 记录左右孩子位置，因此既能直接按字符输入处理，又能很方便地递归输出前序遍历。

4. 数据结构与算法选择

本题用数组模拟结点关系即可。当前实现通过字母减 'a' 得到数组下标，再配合递归函数 PreOrder 遍历，结构非常清晰。

5. 时空复杂度分析

- 时间复杂度： $O(n)$ ，每个结点只访问一次。
- 空间复杂度： $O(n)$ ，主要来自存树结构和递归栈。

代码实现

评测 / 评测记录 / 评测详情

R281462625 记录详情

编程语言C++2

代码长度922B

用时32ms

内存816.00KB

测试点信息源代码

测试点信息

#1 AC 3ms/812.00KB	#2 AC 3ms/788.00KB	#3 AC 3ms/788.00KB	#4 AC 4ms/816.00KB	#5 AC 3ms/812.00KB	#6 AC 3ms/808.00KB	#7 AC 3ms/692.00KB
#8 AC 3ms/788.00KB	#9 AC 3ms/816.00KB	#10 AC 4ms/812.00KB				

jsj0708Baijiahui

所属题目P1305 新二叉树

评测状态Accepted

评测分数100

提交时间2026-06-11 17:46:31

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
typedef struct TreeNode
{
    char val;
    int left;
    int right;
}TreeNode;
TreeNode tree[27];
void PreOrder(int root)
{
    if (root== -1)
    {
        return;
    }
    printf("%c", tree[root].val);
    PreOrder(tree[root].left);
    PreOrder(tree[root].right);
}
int main()
{
    int n = 0;
    scanf("%d", &n);
    for (int i = 0; i < 26; i++)
    {
        tree[i].val = '*';
        tree[i].left = -1;
        tree[i].right = -1;
    }
    char node, lc, rc;
    scanf(" %c %c %c", &node, &lc, &rc);
    int rootIdx = node - 'a';
    tree[rootIdx].val = node;
    if (lc != '*') tree[rootIdx].left = lc - 'a';
    if (rc != '*') tree[rootIdx].right = rc - 'a';
    for (int i = 1; i < n; i++)
    {
        char node, lc, rc;
        scanf(" %c %c %c", &node, &lc, &rc);
        int Idx = node - 'a';
        tree[Idx].val = node;
        if (lc != '*') tree[Idx].left = lc - 'a';

```

```
        if (rc != '*') tree[Idx].right = rc - 'a';
    }
    PreOrder(rootIdx);
    printf("\n");
    return 0;
}
```

(三) 遍历问题

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1229>

知识点：二叉树遍历性质、组合计数

题目简述

给出一棵二叉树的前序遍历和后序遍历，要求输出可能的中序遍历序列总数。

题目保证至少存在一棵二叉树与给定信息相符，且同一序列中不会出现重复字母。

最终答案不超过 $2^{63} - 1$ 。

1.00s, 128MB。

题目分析

1. 读题

和“由中序 + 前序 / 后序唯一确定一棵树”不同，仅有前序和后序时，二叉树结构并不总是唯一。因此题目不是让我们还原某一棵树，而是问：一共可能有多少种中序遍历。

2. 性质观察

造成不唯一的根源在于“某个结点只有一个孩子”时，无法从前序和后序中判断这个孩子到底是左儿子还是右儿子。每出现一次这种情况，就会产生 2 种选择，因此答案的本质仍然是 2^n (单孩子结点个数)。

当前代码采用递归判断：若某段前序中的第二个结点和后序中的倒数第二个结点相同，就说明当前根只有一棵子树，这里会贡献一次模糊选择。

3. 程序设计思路

- 1. 递归处理当前这段前序和后序序列；
- 2. 若当前子树规模小于等于 1，直接返回；
- 3. 判断前序中的第二个结点是否等于后序中的倒数第二个结点；
- 4. 若相等，说明当前根只有一棵子树，计数加一，并递归处理剩余那一整段子树；
- 5. 若不相等，就说明左右子树都存在，此时先分出左子树长度，再递归处理左右两部分；
- 6. 全部递归完成后，输出 2^{count} 。

4. 数据结构与算法选择

本题不需要真的建树，关键在于利用前序与后序的结构关系识别“单孩子结点”。当前实现采用字符串递归拆分的方法来完成计数，因此比单纯枚举相邻字符更贴近二叉树本身的递归结构。

5. 时空复杂度分析

- 时间复杂度：朴素做法为 $O(n^2)$ ，对于字符串长度很小或一般数据范围足够使用。
- 空间复杂度： $O(1)$ 或 $O(n)$ ，取决于是否额外记录位置。

真正的重点不在复杂度，而在遍历性质的理解。

代码实现

平台 / 评测记录 / 评测详情

R281462785 记录详情

编程语言C O2 | 代码长度657B | 用时33ms | 内存1.04MB

测试点信息源代码

测试点信息

#1 AC 3ms/760.00KB	#2 AC 4ms/1.04MB	#3 AC 3ms/812.00KB	#4 AC 3ms/788.00KB	#5 AC 3ms/812.00KB	#6 AC 4ms/816.00KB	#7 AC 3ms/812.00KB
#8 AC 4ms/812.00KB	#9 AC 3ms/832.00KB	#10 AC 3ms/820.00KB				

jsj0708Baijiahui

所属题目P1229 遍历问题

评测状态Accepted

评测分数100

提交时间2026-06-11 17:48:02

```

#include<stdio.h>
#include<string.h>
int count = 0;
void find(char* pre, char* post,int pre_len)
{
    if (pre_len <= 1)
        return;
    char pre_root = pre[1];
    char post_root = post[pre_len - 2];
    if (pre_root == post_root)
    {
        count++;
        find(pre + 1, post, pre_len - 1);
    }
    else
    {
        int left_len = 0;
        while (post[left_len] != pre_root)
        {
            left_len++;
        }
        left_len++;
        find(pre + 1, post, left_len);
        find(pre + 1 + left_len, post + left_len, pre_len - 1 - left_len);
    }
}

int main()
{
    char s1[30];
    char s2[30];
    scanf("%s%s", s1, s2);
    int len = strlen(s1);
    count = 0;
    find(s1, s2, len);
    long long ans = 1LL << count;
    printf("%lld\n", ans);
}

```

(四) 选举学生会

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1271>

知识点：计数排序、桶排序

题目简述

有 m 张选票，每张选票上写着一个候选人编号，候选人编号范围为 1 到 n 。要求将所有选票上的编号按从小到大排序后输出。

其中候选人数 n 很小，但选票总数 m 可以非常大。

对于全部数据， $1 \leq n \leq 999$ ， $1 \leq m \leq 2000000$ 。

1.00s, 128MB。

题目分析

1. 读题

题目表面是在做排序，但数据特征非常明显：待排序数值范围很小，而数据量本身很大。这说明直接使用普通比较排序并不是最有针对性的选择。

2. 性质观察

因为编号只会出现在 1 到 n 之间，所以完全可以统计每个编号出现了多少次。之后再按编号从小到大把它们依次输出若干次，就得到了排序结果。

这正是计数排序或桶排序的典型应用场景。

3. 程序设计思路

当前代码采用的是另一种更直接的写法：

1. 先把全部选票编号读入数组；

- 2. 再调用 `qsort` 对整个数组按从小到大排序；
- 3. 最后顺序输出排序后的结果。

4. 数据结构与算法选择

从题目数据特征来看，计数排序会更有针对性；不过当前实现选择的是“普通数组 + `qsort`”的通用排序方式。这样写代码更短，也同样能够得到正确答案。

5. 时空复杂度分析

- 时间复杂度： $O(m \log m)$ ，当前实现主要耗时在 `qsort` 排序上。
- 空间复杂度： $O(m)$ ，需要保存全部选票编号。

这比通用排序算法更加高效，也更贴合本题的数据特点。

代码实现

首页 / 评测记录 / 评测详情

R281462872 记录详情

编程语言C++2

代码长度466B

用时379ms

内存8.02MB

测试点信息源代码

测试点信息

#1AC3ms/788.00KB

#2AC3ms/808.00KB

#3AC186ms/7.99MB

#4AC4ms/812.00KB

#5AC183ms/8.02MB

jsj0708Baijiahui

所属题目P1271【深基9.例1】选举学生会

评测状态Accepted

评测分数100

提交时间2026-06-11 17:48:46

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int cmp(const void* e1, const void* e2)
{
    int* p1 = (int*)e1;
    int* p2 = (int*)e2;
    return (*p1) - (*p2);
}
int main()
{
    int n = 0;
    int m = 0;
    scanf("%d %d", &n, &m);
    int* arr = (int*)malloc(m * sizeof(int));
    for (int i = 0; i < m; i++)
    {
        int temp = 0;
        scanf(" %d", &temp);
        arr[i]=temp;
    }
    qsort(arr, m, sizeof(int), cmp);
    for (int i = 0; i < m; i++)
    {
        printf("%d ", arr[i]);
    }
    free(arr);
}
```